PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 1
2. Background & Business Objective 2
3. Product Purpose 3
4. System Functionalities 6
5. User Stories & Use Cases 3
6. Features Included (Scope Definition) 3
7. Features Not Included (Scope Control) 3
8. Assumptions & Constraints 4
9. Risks & Questions Throughout Development 4

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: LebihMudah.my - Agentic workflow

Version: 2.0

1. Project Overview

- **Problem Statement:** Traditional property rental platforms are rigid and struggle to process unstructured, messy renter queries effectively. When renters ask specific, multi-step questions that are missing from standard listing databases, traditional systems or basic chatbots hit a dead end and fail to assist the user. This creates a frustrating bottleneck: renters cannot get immediate answers to proceed with a booking, and property owners are burdened with managing fragmented, manual communications to fill in the missing information.
- **Target Domain:** Enabling dynamic and stateful AI orchestration for the property rental market to streamline complex, multi-party interactions between renters and owners while automating the resolution of unstructured queries.
- **Proposed Solution Summary:** With Lebih Mudah.my, people can use messy, natural language searches to find properties and get real-time, context-aware help, eliminating the difficulties of fragmented property hunting. The platform can autonomously stop a user's flow to asynchronously consult property owners about certain missing facts by using a stateful "Agentic Core," and then restart the interaction after the owner responds. This makes it possible for the user to become the recipient of a completely planned, end-to-end rental experience rather than a manual searcher and coordinator of several postings.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective

- **Background of the Problem:** On conventional platforms, the process of renting a house has long followed a strict and linear course. When renters have particular, unstructured questions—like pet regulations or unlisted amenities—that the system is unable to respond to, they are usually forced to explore static databases using limited filters, which frequently leads to a dead end. Currently, resolving these uncertainties requires a significant amount of manual labor since the renter must exit the platform in order to contact the owner through external channels, wait for a response, and then manually try to resume the booking process. Due to the fragmented experience's absence of multi-party coordination and stateful memory, users must act as their own manual coordinators, which hinders the smooth automation of intricate real estate transactions.
- **Importance of Solving This Issue:** Accelerating the transition from a manual search to a confirmed booking by automating complex multi-party coordination, the platform allows renters to resolve specific, unlisted inquiries in minutes instead of days. Breaking the barrier of rigid search filters and structured data entry, any user can express their unique living requirements in plain, unstructured language and receive precise, verified results without the need to manually track or follow up with multiple property owners.

- **Strategic Fit / Impact:** It aligns with enterprise-grade agentic architectures and stateful AI-native development ecosystems. The platform integrates the core Z.AI GLM with frameworks like LangChain or LlamaIndex and a Next.js platform layer to facilitate a developer's workspace as a complete, stateful AI orchestration engine.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Product Purpose

3.1. Main Goal of the System: To provide a stateful, AI-enabled orchestrator for the end-to-end rental journey. It transforms a static search into a dynamic, autonomous conversation that manages everything from discovery to final booking.

3.2. Intended Users (Target Audience):

- Tenants
- Landlords

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. System Functionalities

4.1. Description: The system operates as an agentic AI-powered property rental assistant that transforms unstructured, natural language queries into structured actions. Using a React framework, the system interprets user intent, extracts key parameters, and dynamically orchestrates API tools such as property search, booking, and owner communication.Unlike traditional chatbots, the system supports stateful workflows, allowing it to pause interactions and resume them seamlessly once new information is available. This enables multi-step reasoning, real-time decision-making, and asynchronous coordination between renters and property owners.

4.2. Key Functionalities:

- **Intelligent Property Search:** Translates unstructured user input into structured queries and retrieves matching properties via the search properties tool.
- **Conversational Clarification:** Handles incomplete or ambiguous inputs by interactively asking follow-up questions, ensuring accurate search results and reducing errors.
- **Stateful Workflow Management:** Uses the message owner tool to contact property owners when information is missing. The system pauses and resumes automatically once the owner responds, enabling real-world multi-party coordination.
- **Automated Booking Execution:** Facilitates property booking through the initiate booking tool after validating required inputs such as move-in date and user contact details.

© UMHackathon 2026

4.3. AI Model & Prompt Des

4.3.1. Model Selection: The system uses Z AI GLM as its core model due to its strong capability in handling unstructured natural language and performing multi-step reasoning. This is important because the system needs to not only understand user queries but also decide which actions to take, such as calling APIs for searching properties or contacting owners. Additionally, its ability to generate structured outputs makes it suitable for tool-based execution in an agentic architecture.

4.3.2. **Prompting Strategy:** The system adopts a multi-step agentic prompting strategy using the React framework, where the model follows a structured reasoning flow of Observation, Thought, and Action. This approach ensures that the AI does not simply generate responses but actively decides the next step, such as asking for clarification or invoking a tool. This strategy fits the use case because it enables dynamic decision-making, reduces hallucination, and supports complex workflows

4.3.3. **Context & Input Handling**: The system handles unstructured input through an ingestion router that standardizes all incoming data into a consistent format before processing. It extracts key information such as location, budget, and preferences, and stores conversation context using a session-based memory system. For oversized inputs, the system applies truncation or chunking to ensure only the most relevant information is processed, while preserving the latest user intent for accurate decision-making.

4.3.4. **Fallback & Failure Behavior:** To handle errors or hallucinated responses, the system uses a validation layer that checks all AI-generated actions before execution, preventing invalid API calls. The prompt design also enforces strict rules, such as not guessing missing information, and instead triggering actions like contacting the property owner. If failures occur, the system responds gracefully by asking users to refine their

© UMHackathon 2026

input, and in cases of missing data, it relies on human-in-the-loop interaction to ensure accurate and reliable outcomes.

5. User Stories & Use Cases

- **User Stories:**
  - As a tenant, I want to search for rental properties using natural language so that I do not need to rely only on rigid filters.
  - As a tenant, I want the system to understand incomplete or messy rental requests so that I can still receive relevant property suggestions.
  - As a tenant, I want the system to ask follow-up questions when my request is unclear so that the search results become more accurate.
  - As a tenant, I want the system to contact property owners when information is missing so that I do not need to manually message multiple owners.
  - As a tenant, I want the system to resume my booking flow after receiving owner responses so that the rental process feels continuous and organized.
  - As a landlord, I want to receive structured questions from interested renters so that I can respond clearly and efficiently.
- **Use Cases (Main Interactions):**

  Prompt to UI Flow:

  - User enters a natural language rental request such as “Find me a room near UM under RM900 with parking.”

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Stitch parses the prompt and identifies key rental requirements such as location, budget, property type, parking, move-in date, and preferences.
- The system converts the extracted requirements into structured search parameters.
- Matching rental listings are displayed in the UI.
- If the prompt is incomplete, Stitch asks follow-up questions before showing final results.
- The UI updates dynamically based on the user's answers and refined preferences.

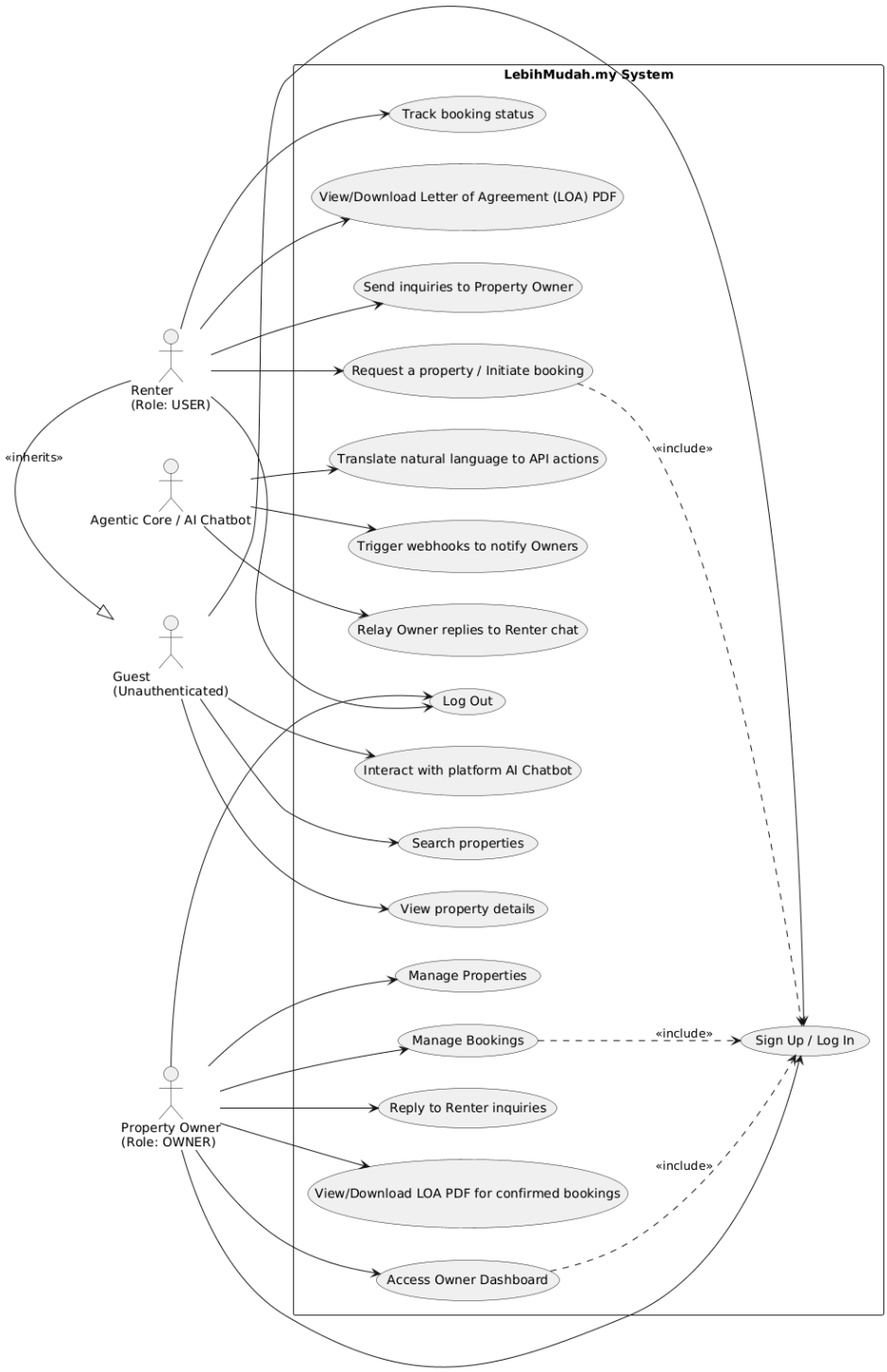
Flow Generation:

- User selects a property or asks the system to continue with a rental option.
- Stitch checks whether all required information is available for the next step.
- If important details are missing, such as deposit amount, pet policy, parking confirmation, or availability, Stitch triggers the owner communication flow.
- The system pauses the user journey while waiting for the owner's response.
- Once the owner replies, Stitch resumes the flow from the correct step.
- The system then generates the next action, such as comparing properties, confirming details, or initiating booking.
- The flow maintains context, so users do not need to restart the rental process manually.

Use Case Diagram:

© UMHackathon 2026

Email: umhackathon@um.edu.my



© UMHackathon 2026

Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- A conversational AI interface that allows users to search for rental properties using natural language.
- Intelligent query understanding that extracts key details (location, budget, preferences) from unstructured input.
- Stateful conversation management that enables the system to pause and resume workflows seamlessly.
- Dynamic tool orchestration, including property search, detail retrieval, booking, and owner communication.
- Asynchronous owner interaction feature, where the system contacts property owners and resumes once a reply is received.

7. Features Not Included (Scope Control)

- ➢ Real payment processing or rental deposit transactions are not included in the prototype.
- ➢ Real legal tenancy agreement generation is not included.
- ➢ Full production-level backend infrastructure is not included.
- ➢ Integration with real property marketplace databases is limited to mock or sample data.
- ➢ Real-time communication with actual landlords may be simulated using mock APIs.
- ➢ Advanced identity verification for tenants and landlords is not included.
- ➢ Multi-user collaboration in the same booking flow is not supported in the initial version.
- ➢ Mobile application development is not included; the prototype focuses on a web-based experience.
- ➢ AI-generated responses are limited to rental assistance and workflow orchestration, not legal or financial advice.

8. Assumptions & Constraints

- **LLM Cost Constraint:** The system assumes moderate token usage per session due to multi-step reasoning and tool interactions. To manage costs, the design

© UMHackathon 2026

Email: umhackathon@um.edu.my

limits unnecessary responses by enforcing structured prompting React and only calling tools when needed, reducing redundant model usage.

- **Technical Constraints:** The system relies on a mock API environment and does not integrate real-world external systems. The architecture is limited to a hackathon-level prototype using simplified tools and simulated workflows.
- **Performance Constraints:** Since the system performs multi-step reasoning and maintains state, response time may increase, especially when handling asynchronous workflows like waiting for owner replies. Real-time performance is dependent on API response speed and model processing.
- **User Input:** The system depends entirely on user-provided input and may require clarification if the input is incomplete or ambiguous. It is not fully autonomous and still relies on human interaction to guide the workflow.

9. Risks & Questions Throughout Development

- **Response Accuracy:** How can we ensure the AI extracts the correct rental requirements from messy user input?
- **Hallucination Risk:** How do we prevent the AI from inventing property details that are not available in the database?
- **Owner Response Delay:** What happens if the property owner does not reply to the system's message?
- **Workflow Reliability:** How can the system safely pause and resume a user's rental journey without losing context?
- **API Failure:** What fallback should be used if the property search, booking, or owner messaging tool fails?
- **User Trust:** How can the system clearly show which information is verified and which information is still pending?
- **Data Privacy:** How should user contact details, preferences, and rental history be protected?
- **Scalability:** Can the system handle many users making multi-step requests at the same time?

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Cost Management:** How can token usage and repeated LLM calls be reduced during long conversations?
- **Prototype Limitation:** Which parts should be simulated for the hackathon demo, and which parts must be fully functional?

© UMHackathon 2026

Email: umhackathon@um.edu.my